```
import pandas as pd
import numpy as np
```

```
df_netflix = pd.read_excel('netflix_clean.xlsx')

df_imdb = pd.read_csv('imdb.csv')
```

```
df_imdb.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71527 entries, 0 to 71526
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   tconst          71527 non-null  object
 1   primaryTitle    71526 non-null  object
 2   startYear       71527 non-null  int64
 3   runtimeMinutes  71527 non-null  object
 4   genres          71527 non-null  object
 5   averageRating   71527 non-null  float64
 6   numVotes        71527 non-null  int64
dtypes: float64(1), int64(2), object(4)
memory usage: 3.8+ MB
```

```
df_netflix.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 868 entries, 0 to 867
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Title         868 non-null    object
 1   Release date  868 non-null    datetime64[ns]
 2   Genre         868 non-null    object
 3   runTime       868 non-null    int64
 4   distributor   868 non-null    object
dtypes: datetime64[ns](1), int64(1), object(3)
memory usage: 34.0+ KB
```

```
df_netflix['runTime'] = pd.to_numeric(df_netflix['runTime'], errors='coerce')
df_imdb['runtimeMinutes'] = pd.to_numeric(df_imdb['runtimeMinutes'], errors='coerce')
```

```
# Use this simplified loop
for index, row in df_netflix.iterrows():
    wiki_title = str(row['Title']).lower().strip()
    wiki_runtime = row['runTime']
    # Add the year from Netflix
    wiki_year = row['Release date'].year

    for i, imdb_row in df_imdb.iterrows():
        imdb_title = str(imdb_row['primaryTitle']).lower().strip()
        imdb_runtime = imdb_row['runtimeMinutes']
        # Add the year from IMDb
        imdb_year = int(imdb_row['startYear'])

        # FIX: Added the '==' back in and included the year check
        if wiki_title == imdb_title and wiki_year == imdb_year:

            # Use a smaller buffer (like 10 mins)
            if abs(wiki_runtime - imdb_runtime) <= 10:


                df_netflix.at[index, 'averageRating'] = imdb_row['averageRating']
                df_netflix.at[index, 'numVotes'] = imdb_row['numVotes']
                df_netflix.at[index, 'tconst'] = imdb_row['tconst']

                print(f"Correctly Linked: {row['Title']}")
                break

print(df_netflix.head())
```

```
Correctly Linked: The Last Mercenary
Correctly Linked: The Kissing Booth 3
Correctly Linked: Beckett
Correctly Linked: Black Island
Correctly Linked: The Secret Diary of an Exchange Student
Correctly Linked: Sweet Girl
Correctly Linked: The Loud House Movie
Correctly Linked: The Witcher: Nightmare of the Wolf
Correctly Linked: He's All That
Correctly Linked: Afterlife of the Party
Correctly Linked: JJ+E
Correctly Linked: Kate
Correctly Linked: Prey
Correctly Linked: Nightbooks
Correctly Linked: Ankahi Kahaniya
Correctly Linked: Confessions of an Invisible Girl
Correctly Linked: Intrusion
Correctly Linked: The Starling
Correctly Linked: Friendzone
Correctly Linked: No One Gets Out Alive
Correctly Linked: Sounds Like Love
Correctly Linked: Forever Rich
Correctly Linked: Swallow
Correctly Linked: The Guilty
Correctly Linked: There's Someone Inside Your House
Correctly Linked: Grudge
Correctly Linked: My Brother, My Sister
Correctly Linked: Bright: Samurai Soul
Correctly Linked: Fever Dream
Correctly Linked: Operation Hyacinth
Correctly Linked: A World Without
Correctly Linked: The Four of Us
Correctly Linked: Night Teeth
Correctly Linked: Stuck Together
Correctly Linked: Little Big Mouth
Correctly Linked: Hypnotic
Correctly Linked: Army of Thieves
Correctly Linked: The Harder They Fall
Correctly Linked: Love Hard
Correctly Linked: Meenakshi Sundareshwar
Correctly Linked: We Couldn't Become Adults
Correctly Linked: Yara
Correctly Linked: Father Christmas Is Back
Correctly Linked: Happiness Ever After
Correctly Linked: Passing
Correctly Linked: 7 Prisoners
Correctly Linked: Red Notice
Correctly Linked: Just Short of Perfect
Correctly Linked: Dhamaka
Correctly Linked: Love Me Instead
Correctly Linked: Tick, Tick... Boom!
Correctly Linked: A Castle for Christmas
Correctly Linked: The Power of the Dog
Correctly Linked: Single All the Way
Correctly Linked: The Whole Truth
```

```
df_netflix.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 868 entries, 0 to 867
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Title          868 non-null    object
 1   Release date   868 non-null    datetime64[ns]
 2   Genre          868 non-null    object
 3   runTime        868 non-null    int64
 4   distributor    868 non-null    object
 5   averageRating  777 non-null    float64
 6   numVotes       777 non-null    float64
 7   tconst         777 non-null    object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 54.4+ KB
```

```
import requests
import pandas as pd
import time

API_KEY = 'aea4facd354ec9322145af2d4b3238c5'
# FIX 1: Added /3 to the end of the URL
BASE_URL = "https://api.themoviedb.org/3"
```

```
def get_tmdb_insights(df_input):
    results = []

    # Trigger words for originality logic
    sequel_terms = ['sequel', 'part ii', 'part 2', 'chapter 2', 'return of', 'vol. 2', 'volume 2']
    remake_terms = ['remake', 'reimagining', 'reboot', 'based on the original']
    adaptation_terms = ['based on the novel', 'based on the book', 'based on the comic',
                        'based on the true story', 'adapted from', 'graphic novel', 'adaptation']

    for index, row in df_input.iterrows():
        imdb_id = row['tconst']
        current_result = {'tconst': imdb_id, 'MPAA_Rating': None, 'is_sequel': 0, 'is_franchise': 0, 'is_adaptation': 0, 'is_re

        # 1. Use 'find' to link IMDb ID to TMDB ID
        find_url = f"{BASE_URL}/find/{imdb_id}?api_key={API_KEY}&external_source=imdb_id"

        try:
            response = requests.get(find_url)

            # FIX 2: Check if the response is actually valid before calling .json()
            if response.status_code != 200:
                print(f"Error {response.status_code} for {row['Title']}. Waiting...")
                time.sleep(2) # Wait a bit if server is busy
                continue

            find_res = response.json()
            movie_results = find_res.get('movie_results', [])

            if not movie_results:
                results.append(current_result)
                continue

            tmdb_id = movie_results[0]['id']

            # 2. Get Details, Keywords, and Release Dates
            details_url = f"{BASE_URL}/movie/{tmdb_id}?api_key={API_KEY}&append_to_response=keywords,release_dates"
            m = requests.get(details_url).json()

            # 3. Extract US MPAA Rating
            mpaa = "Not Rated"
            for country in m.get('release_dates', {}).get('results', []):
                if country['iso_3166_1'] == 'US':
                    for release in country['release_dates']:
                        if release.get('certification'):
                            mpaa = release['certification']
                            break

            # 4. Feature Engineering: Originality Logic (FIXED)
            overview = str(m.get('overview', '')).lower()
            tagline = str(m.get('tagline', '')).lower()
            keywords = " ".join([k['name'].lower() for k in m.get('keywords', {}).get('keywords', [])])
            title_text = str(m.get('title', '')).lower()
            text = f"{overview} {tagline} {keywords}"

            # Make sure 'w' is used consistently in all three lines below:
            is_remake = 1 if any(w in text for w in remake_terms) else 0
            is_adaptation = 1 if any(w in text for w in adaptation_terms) else 0
            is_sequel = 1 if any(w in title_text for w in sequel_terms) or 'sequel' in text else 0
            is_franchise = 1 if (is_sequel == 1 or is_remake == 1) else 0


            current_result.update({
                'MPAA_Rating': mpaa, 'is_sequel': is_sequel, 'is_franchise': is_franchise,
                'is_adaptation': is_adaptation, 'is_remake': is_remake,
                'budget': m.get('budget', 0), 'revenue': m.get('revenue', 0)
            })
            print(f"Matched: {row['Title']} -> {mpaa}")

        except Exception as e:
            print(f"Skipping {row['Title']} due to error: {e}")

        results.append(current_result)
        # FIX 3: Increased sleep slightly to be safe with rate limits
        time.sleep(0.2)

    df_tmdb = pd.DataFrame(results)
    # drop_duplicates prevents the final data from becoming massive if code is run twice
```

```
    df_tmdb = df_tmdb.drop_duplicates(subset=['tconst'])
    return pd.merge(df_input, df_tmdb, on='tconst', how='left')

# RUN THE CODE
data = get_tmdb_insights(df_netflix)
```

```
Matched: Ghost Stories -> NR
Matched: A Fall from Grace -> NR
Matched: Airplane Mode -> NR
Matched: Horse Girl -> R
Matched: To All the Boys: P.S. I Still Love You -> PG-13
Matched: Isi & Ossi -> Not Rated
Matched: The Last Thing He Wanted -> R
Matched: Yeh Ballet -> PG-13
Matched: All the Bright Places -> R
Matched: Guilty -> Not Rated
Matched: Spenser Confidential -> R
Matched: Lost Girls -> R
Matched: Ultras -> Not Rated
Matched: The Occupant -> R
Matched: Maska -> Not Rated
Matched: The Decline -> R
Matched: Uncorked -> Not Rated
Matched: Coffee & Kareem -> R
Matched: The Main Event -> Not Rated
Matched: Love Wedding Repeat -> Not Rated
Matched: Tigertail -> PG
Matched: Earth and Blood -> Not Rated
Matched: Rising High -> Not Rated
Matched: Sergio -> R
Matched: The Willoughbys -> PG
Matched: Time to Hunt -> Not Rated
Matched: Extraction -> R
Matched: Dangerous Lies -> Not Rated
Matched: All Day and a Night -> R
Matched: Mrs. Serial Killer -> Not Rated
Matched: The Half of It -> PG-13
Matched: The Wrong Missy -> R
Matched: The Lovebirds -> R
Matched: Intuition -> Not Rated
Matched: The Last Days of American Crime -> R
Matched: Da 5 Bloods -> R
Matched: A Whisker Away -> PG
Matched: Feel the Beat -> G
Matched: Lost Bullet -> Not Rated
Matched: One-Way to Tomorrow -> Not Rated
Matched: Bulbbul -> Not Rated
Matched: Nobody Knows I'm Here -> Not Rated
Matched: Eurovision Song Contest: The Story of Fire Saga -> PG-13
Matched: Under the Riccione Sun -> Not Rated
Matched: Desperados -> Not Rated
Matched: The Old Guard -> R
Matched: The Players -> R
Matched: Fatal Affair -> Not Rated
Matched: Offering to the Storm -> R
Matched: The Kissing Booth 2 -> PG-13
Matched: Seriously Single -> Not Rated
Matched: Raat Akeli Hai -> Not Rated
Matched: Work It -> PG-13
Matched: Gunjan Saxena: The Kargil Girl -> Not Rated
Matched: Project Power -> R
Matched: Fearless -> PG
Matched: Crazy Awesome Teachers -> Not Rated
Matched: The Crimes That Bind -> Not Rated
```

```
#loading trending movies on netflix
netflix_trending = pd.read_excel('all-weeks-global.xlsx')

print(netflix_trending.info())
```

```
/usr/local/lib/python3.12/dist-packages/openpyxl/styles/stylesheet.py:237: UserWarning: Workbook contains no default style, appl
  warn("Workbook contains no default style, apply openpyxl's default")
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9840 entries, 0 to 9839
Data columns (total 9 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   week                        9840 non-null   object
 1   category                    9840 non-null   object
 2   weekly_rank                 9840 non-null   int64
 3   show_title                  9840 non-null   object
```

```
 4   season_title                4775 non-null   object
 5   weekly_hours_viewed         9840 non-null   int64
 6   runtime                     5760 non-null   float64
 7   weekly_views                5760 non-null   float64
 8   cumulative_weeks_in_top_10  9840 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 692.0+ KB
None
```

```
#having only one column for franchise
data['is_original'] = np.where(
    (data['is_sequel'] == 1) |
    (data['is_franchise'] == 1) |
    (data['is_adaptation'] == 1) |
    (data['is_remake'] == 1),
    'non-original',
    'original'
)
```

```
#preparing the Netflix Top 10 data
weeks_count = netflix_trending.groupby('show_title')['cumulative_weeks_in_top_10'].max().reset_index()
weeks_count.columns = ['Title', 'total_weeks']

#Merging weeks_count into the main 'data' dataframe
data = data.merge(weeks_count, on='Title', how='left')

#Setting 'total_weeks' to 0 for anyone NOT 'Streaming' OR where no match was found
data.loc[data['distributor'] != 'Streaming', 'total_weeks'] = 0
data['total_weeks'] = data['total_weeks'].fillna(0)

#Defining a single function that handles BOTH Cinema and Streaming logic
def classify_movie_success(row):
    #STREAMING LOGIC (Streaming)
    if row['distributor'] == 'Streaming':
        weeks = row['total_weeks']
        if weeks > 4:
            return 'Streaming-hit'
        else:
            return 'Streaming-flop'

    #CINEMA LOGIC (Warner Bros)
    else:
        #Checking if it has budget and revenue data
        if row['budget'] > 0 and not np.isnan(row['revenue']):
            roi = row['revenue'] / row['budget']
            if roi >= 3:
                return 'Cinema-hit'
            else:
                return 'Cinema-flop'
        else:
            # Handle cases with no budget/revenue info
            return 'unknown'

#Creating the final 'performance' column
data['performance'] = data.apply(classify_movie_success, axis=1)
```

```
data.head()
```

	Title	Release date	Genre	runTime	distributor	averageRating	numVotes	tconst	MPAA_Rating	is_sequel	is_franchise
0	Ghost Stories	2020-01-01	Horror anthology	144	Streaming	4.5	6261.0	tt10230404	NR	0	0
1	A Fall from Grace	2020-01-17	Thriller	120	Streaming	5.9	18626.0	tt11390036	NR	0	0
2	Airplane Mode	2020-01-23	Comedy	96	Streaming	5.2	3235.0	tt10731748	NR	0	0
3	Horse Girl	2020-02-07	Psychological drama	104	Streaming	6.0	21793.0	tt11388406	R	0	0
4	To All the Boys: P.S. I Still Love You	2020-02-12	Teen romantic comedy	102	Streaming	6.1	53000.0	tt9354842	PG-13	1	1

```
#exporting to csv
data.to_csv('netflix.csv')
```

Start coding or generate with AI.